

Introduction to R and RStudio

Part 1: Basics Part 2: `tidyverse` (Follow Along
in RStudio) Slides adapted from Carsten Lange

Dr. Jomana Bashatah

Learning Outcomes

What you will learn in this session:

Part 1: R & RStudio Basics

- How to install R and RStudio
- What is the windows layout of RStudio
- How to setup RStudio
- How to create a project (folder) in RStudio
- How to use major functionalities of RStudio
- How to extend R's functionality with R-packages
- Which packages you should install for this course
- Data types and data objects in R
- How very big and very small numbers can be displayed

Part 2: The tidyverse Package

- The Structure of R commands
- About the `tidyverse` package for data frames
 - `select()` and rename columns (variables)
 - `filter()` rows (observations)
 - `mutate()` (define columns (variables); overwrite old or create new)
 - piping (connecting commands) with `|>`.

PART 1: R & RSTUDIO BASICS

Install and Setup R and RStudio

A typical setup to work with R consists of two components:

- **The R Console** which executes R code
- **An integrated development environment (IDE)** such as RStudio

RStudio - Integrated Development Environment (IDE)

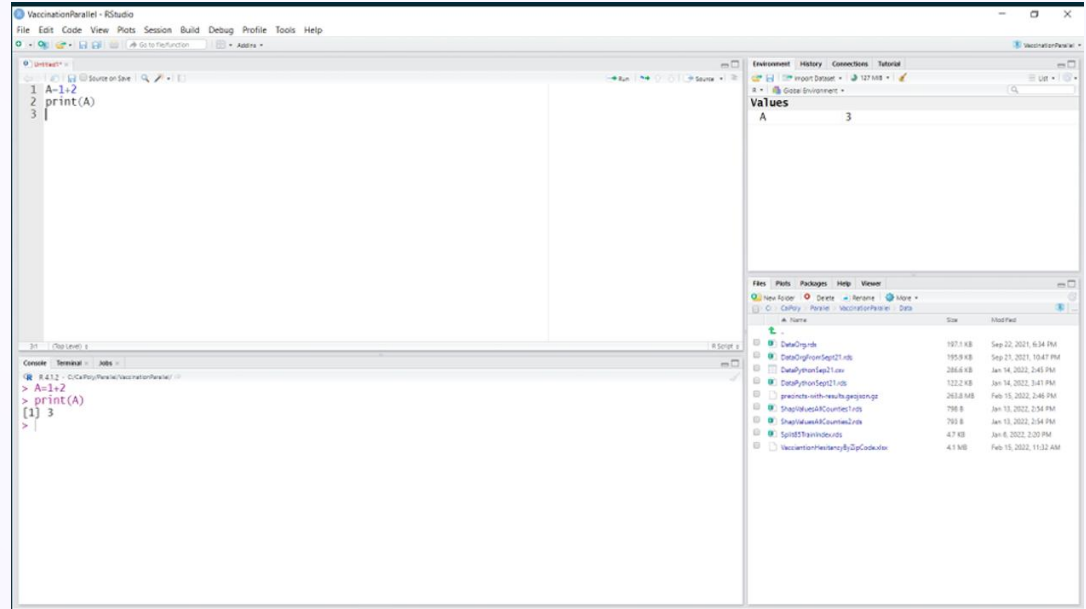
RStudio provides a user-friendly interface with four main panes:

Source Editor (top-left): Write and edit R scripts

Console (bottom-left): Execute R commands directly

Environment/History (top-right): View variables and command history

Files/Plots/Packages/Help (bottom-right):
Manage files, view plots, install packages



RStudio Window

Recommended RStudio Settings

Essential Setup Steps:

1. Do not Restore .RData into workspace at startup:

- Go to Tools → Global Options
- Uncheck “Restore .RData into workspace at startup”

2. Work with R Projects:

- This assigns a directory on your hard drive to your R analysis
- File → New Project

R Packages

R Packages extend R's functionality. They have to be installed only once:

```
# Install packages using Tools -> Install Packages... or:  
install.packages(c("tidyverse", "rio"))
```

After installation they need to be loaded every new R script:

```
library(rio)  
library(tidyverse)
```

Please install these packages soon:

- **tidyverse**: supports easy data processing
- **rio**: allows loading various data resources with one `import()` command
- **janitor**: provides functionality to clean data and rename variable names
- **tidymodels**: streamlines data engineering and machine learning tasks
- **kableExtra**: supports rendering tables in HTML
- **shiny**: needed together with the *learnr* package for interactive exercises

Example: The rio and tidyverse Package

```
library(rio); library(tidyverse)

DataHousing = import("Data/HousingData.csv") |>
  select(Price=price, Sqft=sqft_living,
         Bedrooms=bedrooms, Waterfront=waterfront)

print(DataHousing[1:3,])
```

	Price	Sqft	Bedrooms	Waterfront
1	221900	1180	3	no
2	538000	2570	3	no
3	180000	770	2	no

Note: `import()` requires rio package; `select()` requires tidyverse package

Data Types & Data Objects Overview

Data Types: What can R store?

- **numerical** (num)
- **character** (chr)
- **factor**
- **logic**

Data Objects: What containers does R use?

- **single entry** - single variable
- **list of entries** - vectors
- **table** - dataframe and tibble
- **advanced objects** - plots, models, prediction results

Data Types

Main

- **Numerical Data Type (num):** Numerical values (e.g., 1, 523, 3.45) used for calculation. ZIP-Codes are NOT numerical data type.
- **Character Data Type (chr):** Storing sequence of characters, numbers, and/or symbols (e.g. first/last names, street addresses, Zip-codes)
- **Factor Data Type (factor):** A factor is an R data type that stores categorical data in an effective way. factor data types are also required by many classification models in R.
- **Logic Data Type (logic):** A data type that stores the logic states TRUE and FALSE is called a logic object (sometimes called Boolean)

Truth Table

Numerical

Numerical values are used for calculations (therefore ZIP-Codes are not numerical).
Numerical data can be discrete (integer) or continuous (double).

```
A = as.integer(2)
B = as.integer(3)
str(A) # str() returns structure of a variable
      int 2
C = 1.23
str(C)
      num 1.23
print(A * C)
[1] 2.46
A^B
[1] 8
A/B # Returns num type
[1] 0.6666667
```

Character

Note: What is called a “character” in R is often called a “string” in other programming languages.

Character data types must be surrounded by quotes:

```
MyText = "Hello world!"  
print(MyText)  
[1] "Hello world!"
```

Character variables can be concatenated with the `cat()` command:

```
FirstName = "Jomana"  
LastName = "Bashatah"  
cat(FirstName, LastName) # R adds a space  
automatically  
Jomana Bashatah
```

Factor

A factor stores categorical data efficiently. Categorical data are character type data covering a few categories such as hair color (blonde, brown, red, black). They can be coded with numbers (e.g., 1-5 for hair color) and use less memory.

```
People=tribble(~Name,~Gender,  
               "John", "male",  
               "Jane", "female",  
               "Mia", "female",  
               "Brid", "female",  
               "Greg", "male")
```

```
print(People)  
# A tibble: 5 × 2  
  Name  Gender  
  <chr> <chr>  
1 John  male  
2 Jane  female  
3 Mia   female  
4 Brid  female  
5 Greg  male
```

Logic

Logic variables store TRUE and FALSE. They can be combined with and/or. Internally TRUE is stored as 1 and FALSE is stored as 0.

```
IsConcertGood = FALSE
IsCompanyGood = TRUE
cat("Is the concert good?", IsConcertGood, "Is the company good?",
IsCompanyGood)
Is the concert good? FALSE Is the company good? TRUE
IsEveningAmazing=IsConcertGood & IsCompanyGood
cat("Is the evening amazing?", IsEveningAmazing)
Is the evening amazing? FALSE
IsEveningGood = IsConcertGood | IsCompanyGood # | stands for "or"
cat("Is the evening good?", IsEveningGood)
Is the evening good? TRUE
IsConcertGood+IsCompanyGood+17
[1] 18
```

Truth Table

R Object A	R Object B	A & B	A B
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

Data Objects

Main

- **Single Value Object:** Stores just one value
- **Vector Object:** Stores a list of values
- **Data Frame (Tibble) Object:** Table-like structure
- **List Object:** (not covered in this course)
- **Advanced Objects:** plots, models, recipes

Single Value Object

Object stores a single value:

```
A = 123.768
```

```
B = 3
```

```
C = "Hello World"
```

```
IsLifeGood = TRUE
```

Vector Objects

A vector object stores a list of values (numerical, character, factor, or logic).

Example: Weather during the last three days in Stattuon:

```
VecTemp = c(70, 68, 55)
VecWindSpeed = c("low", "low", "high")
VecIsSunny = c(TRUE, TRUE, FALSE)
```

Vector objects can be used as arguments for an R command to calculate

```
MeanForecTemp=mean(VecTemp)
cat("The average forecasted temperature is", MeanForecTemp)
The average forecasted temperature is 64.33333
ForecDays=length(VecTemp)
cat("The forecast is for", ForecDays, "days.")
The forecast is for 3 days.
```

Data Frames (Tibbles)

A data frame is similar to an Excel table:

Survived	Pclass	Gender	Age	FareInPounds
0	3	Male	22	7.2500
1	1	Female	38	71.2833
1	3	Female	26	7.9250
1	1	Female	35	53.1000
0	3	Male	35	8.0500

Extracting Vectors and Performing Calculations

Numerical Vectors

```
VecFareInPounds = DataTitanic$FareInPounds
AvgFare = mean(VecFareInPounds)
cat("The average fare of Titanic
passengers was:", AvgFare, "British
Pounds")
```

The average fare of Titanic passengers
was: 32.30542 British Pounds

Extracting Vectors and Performing Calculations

Logical Vectors

```
DataTitanic$Survived = as.logical(DataTitanic$Survived)
str(DataTitanic)
'data.frame':   887 obs. of  8 variables:
 $ Survived      : logi  FALSE TRUE TRUE TRUE FALSE FALSE ...
 $ Pclass        : int   3 1 3 1 3 3 1 3 3 2 ...
 $ Name          : chr   "Mr. Owen Harris Braund" "Mrs. John Bradley
Cumings" "Miss. Laina Heikkinen" "Mrs. Jacques Heath Futrelle" ...
 $ Gender        : chr   "male" "female" "female" "female" ...
 $ Age           : num   22 38 26 35 35 27 54 2 27 14 ...
 $ SiblingsSpousesAboard: int  1 1 0 1 0 0 0 3 0 1 ...
 $ ParentsChildrenAboard: int  0 0 0 0 0 0 0 1 2 0 ...
 $ FareInPounds   : num   7.25 71.28 7.92 53.1 8.05 ...
SurvRate = mean(DataTitanic$Survived)
cat("The average survival rate of Titanic passengers was:", SurvRate)
The average survival rate of Titanic passengers was: 0.3855693
```

Data Frames vs. Tibbles

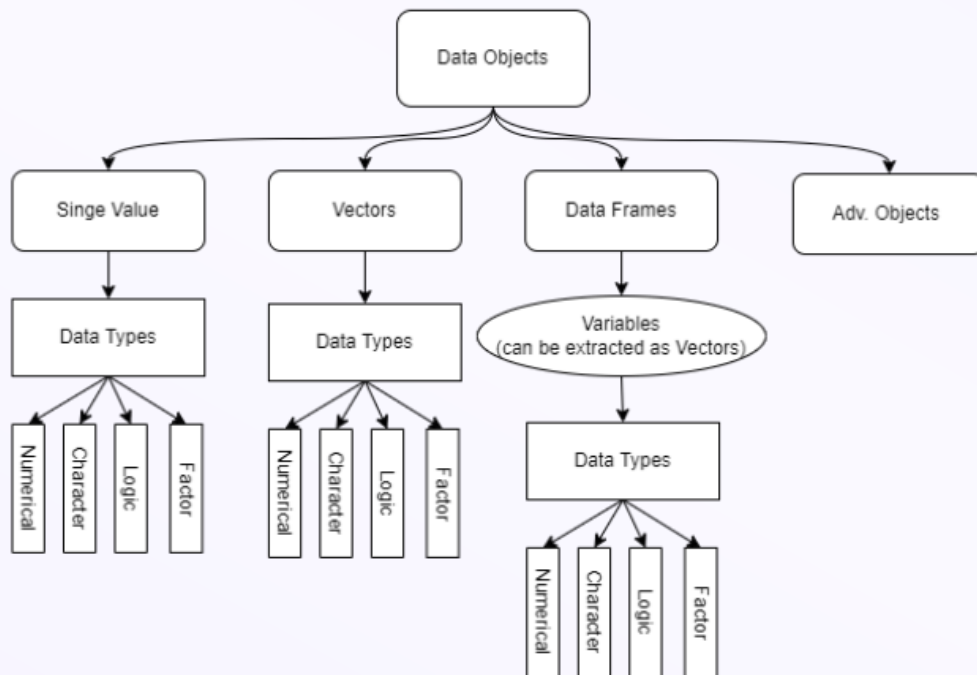
A tibble is a more advanced sub-type of a data frame. Convert with `as_tibble()`:

Key Differences:

A few of the differences between data frames and tibbles:

1. A data frame outputs all its rows and columns by default. A tibble outputs only the first 10 rows and the variables that fit on the screen but provides information about omitted variables and rows.
2. A data frame can have row names, while a tibble cannot.
3. In R version <4.1 a data frame converts all character values to factor type. This conversion was often confusing and annoying. In contrast, a tibble only coerces character values into factor on demand. Since R version 4.1 regular data frames behave the same as tibbles.

Summary: Data Types and Objects



How Are Very Big Numbers Presented

The GDP for 2021 in the US was \$22,996,086,000,000 (rounded to millions)

$$GDP = 2.2996086 \cdot 10000000000000$$

$\Leftrightarrow$

$$GDP = 2.2996086 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10$$

$\Leftrightarrow$

$$GDP = 2.2996086 \cdot 10^{13}$$

What R does:

```
GDPUS = 22996086000000
```

```
print(GDPUS)
```

```
[1] 2.299609e+13
```


How Are Very Small Numbers Presented

main

The probability of getting struck by lightning in the US is about 0.000000000365 on any randomly chosen day.

$$\begin{aligned} ProbLight &= \frac{3.65}{10000000000} \\ &\Leftrightarrow \\ ProbLight &= \frac{3.65}{10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10} \\ &\Leftrightarrow \\ ProbLight &= \frac{3.65}{10^{10}} \\ &\Leftrightarrow \\ ProbLight &= 3.65 \cdot 10^{-10} \end{aligned}$$

Why?

In the U.S., a person has a 1:10,000-lifetime risk of being struck by lightning. Assuming a life span of 75 years and 365.25 days per year, the probability per day is:

$$\begin{aligned} \frac{1}{10,000 \cdot 365.25 \cdot 75} &= \frac{1}{273,937,500} \\ &= 0.000000000365 \\ &= 3.65 \times 10^{-10} \end{aligned}$$

What R Does

```
ProbStruck = 0.000000000365
cat("Probability to get struck by lightning on an avg. day:", ProbStruck)
Probability to get struck by lightning on an avg. day: 3.65e-10
```

PART 2: THE TIDYVERSE PACKAGE

Basics of R Commands

R commands consists of the **command's name followed by a pair of parentheses**:

```
command()
```

...

Inside the `()` we can define one or more **arguments** for the command.

...

```
VecTest=c(1, 2, 3)
```

```
sum(x=VecTest)
```

```
[1] 6
```

```
mean(VecTest)
```

```
[1] 2
```

- Arguments in a command usually have names such as `x=` or `data=`
- R does not require to use the argument's name, but **order matters**
- R commands have many arguments. Most have default values
- We can nest commands. However, nesting too deeply makes code difficult to read.

Structure of R Commands

Most R commands have the following structure:
Often the `data` argument is the first argument in a command. Usually named `data=` or `x=`.

Use a Command with and without Argument Names

```
VecTest=c(1,2,3)
```

```
Result=mean(x=VecTest, trim=0, na.rm=FALSE)
cat("The mean of the values in vector VecTest is:", Result)
The mean of the values in vector VecTest is: 2
```

...

```
Result=mean(VecTest, 0, FALSE)
cat("The mean of the values in vector VecTest is:", Result)
The mean of the values in vector VecTest is: 2
```

...

```
Result=mean(VecTest)
cat("The mean of the values in vector VecTest is:", Result)
The mean of the values in vector VecTest is: 2
```

...

All three examples are equivalent

Try ? mean in the Rstudio console to see the default values.

Important Commands from `tidyverse/dplyr` Package

- `dplyr` package is part of the `tidyverse` (meta) package
- `library(tidyverse)` (loads the `tidyverse` and its packages)
- `select()` selects columns (variables) from a data frame
- `filter()` filters rows (observations) for specific criteria
- `mutate()` calculates new or overwrites existing columns (variables) based on other columns (just like Excel).

Titanic Dataset

```
library(rio)
DataTitanic=import("Data/Titanic.csv")
head(DataTitanic)
```

	Survived	Pclass	Name	Gender	Age	SiblingsAboard	SpousesAboard
1	0	3	Mr. Owen Harris Braund	male	22		1
2	1	1	Mrs. John Bradley Cumings	female	38		1
3	1	3	Miss. Laina Heikkinen	female	26		0
4	1	1	Mrs. Jacques Heath Futrelle	female	35		1
5	0	3	Mr. William Henry Allen	male	35		0
6	0	3	Mr. James Moran	male	27		0

	ParentsAboard	ChildrenAboard	FareInPounds
1		0	7.2500
2		0	71.2833
3		0	7.9250
4		0	53.1000
5		0	8.0500
6		0	8.4583

The select () Command

- `select(DataMine, Var1, Var2)` **selects columns (variables) Var1 and Var2 from a data frame DataMine. The first argument is the data= argument followed by the names of the selected variables.**
- `select(Data, -Var1, -Var2)` **selects all columns (variables) except Var1 and Var2 from a data frame DataMine.**

...

Here is an example using the DataTitanic data frame from the previous slide:

```
library(tidyverse)
DataTitanicSelVar=select(DataTitanic, Survived, Name, Gender, Age)
head(DataTitanicSelVar)
```

	Survived	Name	Gender	Age
1	0	Mr. Owen Harris Braund	male	22
2	1	Mrs. John Bradley Cumings	female	38
3	1	Miss. Laina Heikkinen	female	26
4	1	Mrs. Jacques Heath Futrelle	female	35
5	0	Mr. William Henry Allen	male	35
6	0	Mr. James Moran	male	27

The `filter()` Command

The `filter()` command filters rows (observations) of a data frame for specific criteria. The first argument is the `data=` argument followed by the filter criteria.

...

E.g., *filter* for female passengers from the dataset: Use `DataTitanicSelVar` that we created in the previous slide (note that we have to use `==` instead of `=` for the criteria):

```
DataTitanicSelVarFem=filter(DataTitanicSelVar, Gender=="female")  
head(DataTitanicSelVarFem)
```

	Survived	Name	Gender	Age
1	1	Mrs. John Bradley Cumings	female	38
2	1	Miss. Laina Heikkinen	female	26
3	1	Mrs. Jacques Heath Futrelle	female	35
4	1	Mrs. Oscar W Johnson	female	27
5	1	Mrs. Nicholas (Adele Achem) Nasser	female	14
6	1	Miss. Marguerite Rut Sandstrom	female	4

The mutate () Command

`mutate()` creates or overwrites columns (variables) based on other columns (just like Excel). The first argument is the `data=` argument followed by the instructions on how to create the new variable.

...
E.g., *mutate* calculates new column `Born` based on `Age` during Titanic disaster (1912). Uses `DataTitanicSelVarFem` from previous slide:

```
DataTitanicSelVarFemBirthYear=mutate(DataTitanicSelVarFem, Born=1912-Age)  
head(DataTitanicSelVarFemBirthYear)
```

	Survived	Name	Gender	Age	Born
1	1	Mrs. John Bradley Cumings	female	38	1874
2	1	Miss. Laina Heikkinen	female	26	1886
3	1	Mrs. Jacques Heath Futrelle	female	35	1877
4	1	Mrs. Oscar W Johnson	female	27	1885
5	1	Mrs. Nicholas (Adele Achem) Nasser	female	14	1898
6	1	Miss. Marguerite Rut Sandstrom	female	4	1908

Summary

1. We selected variables *Survived, Name, Gender, Age* and saved in `DataTitanicSelVar`
2. We filtered for females and saved in `DataTitanicSelVarFem`
3. We mutated to calculate new variable and saved finally in `DataTitanicSelVarFemBirthYear`

...

Could this be done easier?

Note, overwriting data frames such as `DataTitanic` is usually a bad idea!

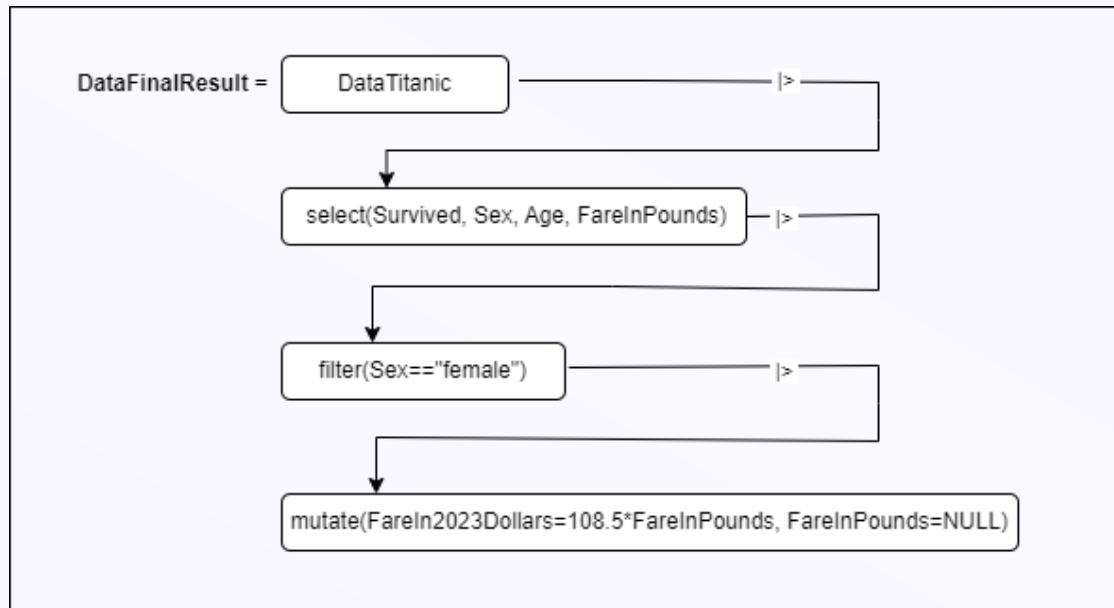
Alternative: Nesting

(I am not serious)

```
library(tidyverse)
DataTitanicFinal= mutate(
  filter(select(DataTitanic, Survived, Name, Gender, Age),
    Gender=="female"),
  Born=1912-Age)
head(DataTitanicFinal)
```

	Survived	Name	Gender	Age	Born
1	1	Mrs. John Bradley Cumings	female	38	1874
2	1	Miss. Laina Heikkinen	female	26	1886
3	1	Mrs. Jacques Heath Futrelle	female	35	1877
4	1	Mrs. Oscar W Johnson	female	27	1885
5	1	Mrs. Nicholas (Adele Achem) Nasser	female	14	1898
6	1	Miss. Marguerite Rut Sandstrom	female	4	1908

Piping Schema



Piping Schema

Alternative: Piping

(will be used throughout the course/book)

```
library(tidyverse)
DataTitanicFinal= DataTitanic |>
  select(Survived, Name, Gender, Age) |>
  filter(Gender=="female") |>
  mutate(Born=1912-Age)
head(DataTitanicFinal)
```

	Survived	Name	Gender	Age	Born
1	1	Mrs. John Bradley Cumings	female	38	1874
2	1	Miss. Laina Heikkinen	female	26	1886
3	1	Mrs. Jacques Heath Futrelle	female	35	1877
4	1	Mrs. Oscar W Johnson	female	27	1885
5	1	Mrs. Nicholas (Adele Achem) Nasser	female	14	1898
6	1	Miss. Marguerite Rut Sandstrom	female	4	1908

Questions